CASE STUDY · HTK 0396241674

HTK Match

Workspace sàng CV có thể kiểm chứng: xếp hạng theo kỹ năng kèm giải thích, golden set nhãn 0/1/2, precision@5 – recall@10 – MRR và cổng chặn hồi quy.

Next.js 15 · React 19 · TypeScript · PGlite · Vercel Blob · Vitest

Tác giả: HTK 0396241674
Ngày: 11/11/2026
Phạm vi: dự án 08/10 trong portfolio full-stack — nhóm Commercial SaaS & Dev Tools
Live: https://htk-resume-matcher.vercel.app

Tài liệu gồm 6 trang: bài toán, kiến trúc và mô hình chấm điểm, quy trình đánh giá, đánh đổi kèm số đo thật, và bài học sau khi triển khai.

1. Bài toán

Một recruiter nhận 100–300 CV cho mỗi vị trí. Công cụ phổ biến là lọc từ khoá hoặc một mô hình AI không giải thích được. Cả hai đều có cùng vấn đề: không ai biết chất lượng xếp hạng đang tăng hay giảm sau mỗi lần đổi từ khoá, trọng số hay nhà cung cấp mô hình.

HTK Match chọn hướng ngược lại: dùng một bộ chấm điểm đơn giản, tất định, giải thích được từng dòng; đặt nó lên một golden set do con người chấm; và biến kết quả đo thành một cổng chặn hồi quy. Mục tiêu không phải "AI tuyển người" mà là *làm cho quyết định sàng lọc có thể kiểm tra lại*.

Bốn kiểu thất bại cần chặn

1. **Hộp đen.** Ứng viên bị loại nhưng recruiter không biết vì thiếu kỹ năng nào, thiếu bao nhiêu năm hay do nhiễu văn bản.
2. **Đổi thuật toán mù.** Thêm một alias kỹ năng có thể đẩy một nhóm ứng viên lên mà không ai đo được tác động.
3. **Tự khen.** Nếu dùng chính feedback "phù hợp" để tính điểm chất lượng, hệ thống luôn đạt điểm cao dù xếp hạng sai.
4. **Rò dữ liệu.** CV là dữ liệu cá nhân; gửi toàn bộ nội dung sang API bên thứ ba chỉ để xếp hạng là đánh đổi không cần thiết.

Yêu cầu và ràng buộc

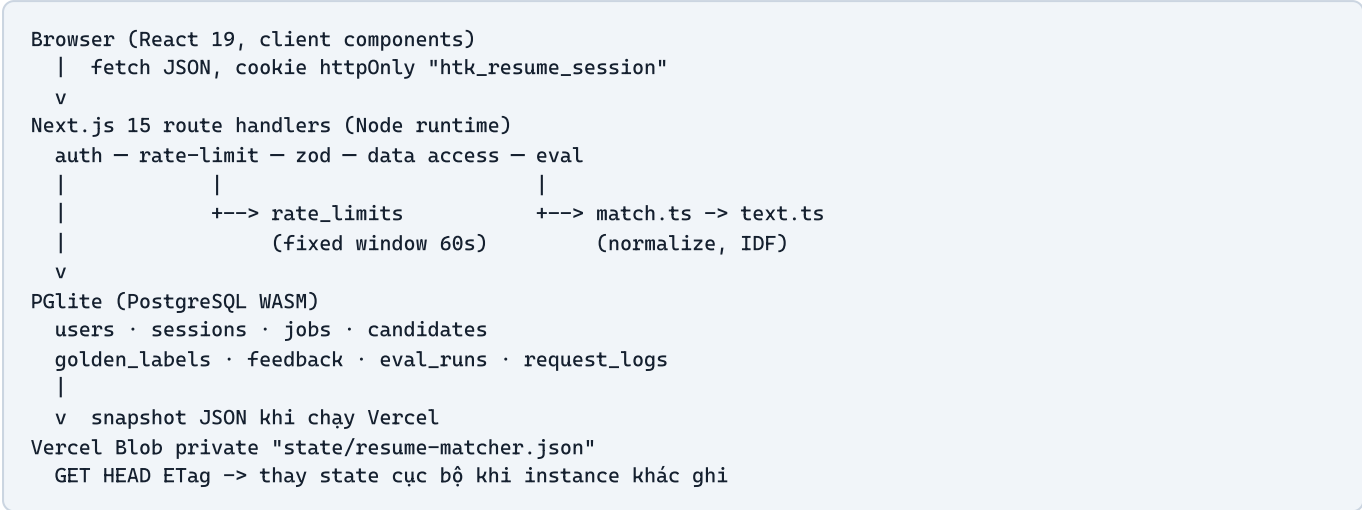
Yêu cầu	Cách hiện thực
Xếp hạng giải thích được	Ba thành phần điểm rời, kỹ năng khớp/thiếu, lý do seniority và IDF cho từng ứng viên.
Đo được chất lượng	Nhãn vàng 0/1/2, precision@5, recall@10, MRR; lưu mọi lần chạy.
Chặn hồi quy	Ngưỡng 0,60 / 0,90 / 0,80; fail rõ trên dashboard và trong lịch sử.
Tách phép đo khỏi cá nhân hoá	Gate chấm ranking thuần; feedback chỉ cộng/trừ 0,12 khi hiển thị.
Không gửi CV ra ngoài	Toàn bộ xử lý trong Next.js; không gọi dịch vụ AI/embedding.
Nhiều tài khoản	Mọi truy vấn lọc theo `user_id`; job của người khác trả 404.

Tiêu chí thành công

- Recruiter nhìn một dòng biết ngay vì sao ứng viên đứng vị trí đó và thiếu gì để lên.
- Một thay đổi làm precision@5 tụt dưới 60% phải bị phát hiện trong cùng lần chạy đánh giá.
- Upload một CV văn bản xong, ứng viên xuất hiện trong mọi bảng xếp hạng mà không cần nhập tay kỹ năng.
- Dashboard nêu rõ số nhãn, số feedback và trạng thái cổng, không dùng số liệu ước lượng.

> Phạm vi dữ liệu là 3 vị trí × 12 CV × 36 nhãn trong bản demo. Đây là quy mô đủ để chứng minh quy trình đo, không phải tuyên bố về độ chính xác trên thị trường tuyển dụng thật.

2. Giải pháp — kiến trúc và mô hình chấm điểm



Mọi request đi qua một wrapper chung: sinh `x-request-id`, bắt lỗi thành JSON `{ error: { code, message } }`, ghi log dòng JSON và bảng `request_logs`. Các endpoint nhạy cảm (login, upload CV, nhãn, eval) gọi rate limit trước khi đọc body nghiệp vụ.

Công thức điểm

Với mỗi cặp job–ứng viên, điểm nằm trong khoảng 0..1:

- **0,5 × phủ kỹ năng** — tỉ lệ kỹ năng bắt buộc có mặt, sau khi chuẩn hoá alias.
- **0,32 × trùng từ khoá IDF** — phần khối lượng từ của job (mô tả, tiêu đề, kỹ năng nhân ba) xuất hiện trong CV; từ hiếm có trọng số cao hơn.
- **0,18 × mức kinh nghiệm** — `min(1, years / minYears)`; thiếu năm bị phạt tuyến tính.
- **±0,12 feedback** — chỉ áp dụng khi trả ranking cho recruiter, không áp dụng trong eval.

Kỹ năng được nhận diện theo token trọn vẹn, không phải substring: `ts` không khớp "tests", `node.js` khớp "Node.js" và alias "nodejs", nhưng "reactive" không bị tính là React. Số năm lấy từ trường có cấu trúc, nếu trống mới đọc câu "6 năm kinh nghiệm".

Vì sao tất định thay vì embedding

Tiêu chí	So khớp từ vựng + IDF	Embedding / API AI
Kết quả lặp lại	Có, cùng input → cùng điểm	Phụ thuộc model, version, tham số
Dữ liệu CV	Không rời khỏi server	Gửi sang bên thứ ba
Giải thích	Chỉ ra đúng kỹ năng và token đóng góp	Khó quy trách nhiệm từng phần
Ngữ nghĩa	Yếu: "backend thiếu TypeScript" có thể xếp dưới "frontend có TypeScript"	Tốt hơn nhưng vẫn cần golden set để kiểm tra

Chọn vế trái vì bài toán của dự án là *đo lường*, không phải tối đa hoá độ thông minh. Hàm xếp hạng nằm gọn trong `lib/match.ts`; thay bằng embedding không đổi golden set, gate hay lịch sử eval.

3. Giải pháp — đánh giá, cổng hồi quy và vòng phản hồi

Luồng một lần đánh giá
chọn job có nhãn → xếp hạng toàn bộ CV (không feedback) → cắt top 5 và top 10 → tính P@5, R@10, MRR theo từng job → lấy trung bình theo job → so ngưỡng → lưu `eval_runs.detail`.

Định nghĩa ba chỉ số

- **Precision@5** = số ứng viên có nhãn ≥ 1 trong top 5 / min(5, số ứng viên). Đo top 5 có bị nhiễu không.
- **Recall@10** = số ứng viên liên quan trong top 10 / tổng số ứng viên liên quan. Đo có bỏ sót người tốt không.
- **MRR** = 1 / vị trí của ứng viên liên quan đầu tiên. Đo người tốt nhất có bị đẩy xuống không.

Ba chỉ số bổ sung nhau: precision cao nhưng recall thấp là bỏ sót; recall cao nhưng MRR thấp là tìm thấy nhưng xếp sai thứ tự. Ngưỡng hiện tại **0,60 / 0,90 / 0,80**, thấp hơn baseline một khoảng để bắt được tụt giảm thật thay vì fail vì dao động nhỏ.

Golden set và chống tự khen

Mỗi job có nhãn 0/1/2 cho toàn bộ 12 ứng viên. Nhãn 0 được lưu thành hàng thật, nên precision phạt được cả trường hợp xếp người không liên quan vào top 5. Feedback "phù hợp / không phù hợp" là tín hiệu riêng của recruiter, được cộng/trừ 0,12 khi xem ranking nhưng bị loại khỏi đầu vào của eval. Nếu gộp hai thứ, hệ thống sẽ tự khen chính lần bấm của mình.

Ba lớp chống lỗi dữ liệu

- **Zod ở biên** — giới hạn độ dài tên, mô tả, CV 20.000 ký tự, số năm 0–40; sai trả 400 kèm thông điệp tiếng Việt.
- **Kiểm tra quyền sở hữu** — job/CV/nhãn/feedback đều lọc theo user; người khác nhận 404 thay vì 403 để không lộ sự tồn tại.
- **Quy tắc xoá** — CV đang có nhãn vàng không xoá được (409); job xoá kèm nhãn và feedback của chính job đó.

Bề mặt API

```
GET  /api/health                 trạng thái DB và chế độ lưu trữ
POST /api/auth/login|register    session cookie httpOnly, rate limit
GET  /api/jobs                   danh sách job + số nhãn
POST /api/jobs                   tạo job, tách kỹ năng
GET  /api/jobs/:id/ranking       điểm, breakdown, lý do, nhãn, feedback
POST /api/jobs/:id/feedback      relevant | not_relevant | null
POST /api/jobs/:id/labels        nhãn vàng 0 | 1 | 2 | null
GET  /api/candidates             thư viện CV
POST /api/candidates             upload văn bản, tự tách kỹ năng/năm
GET  /api/eval                   lịch sử lần chạy
POST /api/eval                   chạy gate, lưu kết quả
GET  /api/metrics                đếm job/CV/nhãn/feedback, p50/p95 request
```

UI có ba trạng thái bắt buộc: đang tải ("Đang tải workspace…"), rỗng (chưa có job/CV/lịch sử) và lỗi (banner đỏ kèm nút tải lại). Các hành động đổi dữ liệu đều khóa nút theo từng ứng viên để tránh bấm đúp, và thông báo kết quả nằm trong vùng `aria-live`.

> Giới hạn có chủ ý: recall@10 gần bằng recall toàn pool vì mỗi job chỉ có 12 ứng viên. Golden set nhỏ giúp demo nhanh, nhưng chưa đủ để kết luận về khả năng xếp hạng trên hàng trăm CV.

4. Đánh đổi, số đo thật và điểm chưa hoàn thiện

Ba đánh đổi chính

Đánh đổi	Được	Mất
Từ vựng IDF thay vì AI	Tất định, offline, giải thích được, CV không rời server	Không hiểu ngữ nghĩa và phủ định; alias phải bảo trì tay
PGlite + Blob snapshot	PostgreSQL thật, không dịch vụ ngoài, deploy Vercel một lần	Một writer, đồng bộ eventual giữa các lambda, không phù hợp tải ghi lớn
Feedback tách khỏi gate	Không tự khen; đo được chất lượng thuần của thuật toán	Recruiter phải chấm nhãn vàng riêng, thêm một bước thủ công

Số đo trên máy local

Windows, Node 24, `next dev`; 40 request xếp hạng và 6 lần chạy eval. Đây là số của môi trường phát triển một máy, chưa phải benchmark production.

Phép đo	Kết quả	Ghi chú
Precision@5 baseline	**0,6667**	Ngưỡng 0,60 — đạt
Recall@10 baseline	**1,0000**	Ngưỡng 0,90 — đạt
MRR baseline	**1,0000**	Ngưỡng 0,80 — đạt
GET ranking (12 CV)	p50 36,2 ms · p95 45,5 ms	Gồm truy vấn PGlite và JSON
POST eval (3 job × 12 CV)	p50 47,0 ms · p95 54,5 ms	Tính điểm, gate và ghi lịch sử
Login sai liên tiếp	10 × 401 rồi 2 × 429	Cửa sổ 5 phút theo email + IP
Test tự động	**30/30 pass**	Text, match, eval, tích hợp PGlite
Tràn ngang 375 / 1440	**0 px**	Workspace và trang xếp hạng

0,6667
Precision@5 baseline — thấp hơn 1 vì top 5 job frontend chứa hai ứng viên backend có TypeScript.

30/30
Test pass, gồm 11 test tích hợp chạy PGlite trong thư mục tạm.

Những chỗ còn yếu, nói thẳng

- **Hiểu sai phủ định.** "Chưa từng dùng Kubernetes" vẫn bị tính là có kỹ năng đó. Cần phân đoạn câu hoặc mô hình NER để sửa.
- **Bỏ sót CV không có từ khoá.** CV mô tả "xây hệ thống thanh toán" mà không ghi "Node.js" sẽ mất điểm phủ kỹ năng dù kinh nghiệm tương đương.
- **Golden set nhỏ.** 12 ứng viên/job khiến recall@10 gần như luôn bằng 1; chưa đủ để phát hiện lỗi xếp hạng ở đuôi dài.
- **Chưa parse PDF/DOCX.** Chỉ nhận văn bản thuần; file nhị phân bị từ chối ở phía trình duyệt.
- **Rate limit theo instance.** Serverless nhiều instance có thể vượt hạn; production cần Redis hoặc cửa sổ dùng chung.

> Điểm mạnh của dự án không nằm ở thuật toán thông minh nhất, mà ở chỗ mọi thay đổi đều có một con số chống lại nó: golden set, ba chỉ số và một cổng fail rõ ràng.

5. Bài học

Đo trước khi thông minh hơn. Việc đầu tiên nên làm không phải thêm embedding mà là viết golden set và ba chỉ số. Khi đã có thước đo, việc thay thuật toán trở thành một thí nghiệm có kết luận thay vì một niềm tin.

Tách tín hiệu cá nhân hoá khỏi tín hiệu đánh giá. Feedback là cách recruiter sửa hành vi hệ thống, nhưng nếu dùng nó để chấm chính hệ thống thì kết quả luôn đẹp. Hai đường ống riêng, một cờ `withFeedback`, đủ để giữ sự trung thực.

Giải thích là một phần của API, không phải trang trí. Trả về `matchedSkills`, `missingSkills` và breakdown giúp UI nói được "vì sao" mà không phải suy đoán lại từ điểm tổng. Nó cũng buộc mô hình chấm điểm phải có thành phần rõ ràng.

Viết test tích hợp cho đúng chỗ dễ vỡ. Phần scoring có test đơn vị, nhưng bug thật thường nằm ở ghép dữ liệu: nhãn của job này với ranking của job khác, xoá CV đang có nhãn, người dùng A đọc job của B. 11 test tích hợp PGlite bắt được nhóm lỗi đó nhanh hơn test UI.

Demo phải nói cả phần chưa làm. Ghi rõ recall@10 chưa có ý nghĩa vì pool nhỏ, và "chưa từng dùng X" vẫn bị tính là có X, khiến người đọc tin phần còn lại hơn. Một case study trung thực có giá trị hơn một bảng số toàn màu xanh.

> Hướng tiếp theo nếu có thêm thời gian: parse PDF/DOCX, mở rộng pool lên 50–100 CV/job để recall@10 có ý nghĩa, thêm chỉ số NDCG cho nhãn 2, và chuyển rate limit sang Redis khi chạy nhiều instance.